6
Amazon Machine Learning Services

We discussed cloud databases and big data analytics in previous chapters. Part of the data analytics spectrum, **machine learning** (**ML**) involves building models or algorithms that enable computers to analyze and learn from data, identify patterns, relationships, and trends that can be used to make predictions or decisions.

Cloud-based ML platforms provide a range of tools and services to support ML workflows of data preparation, feature engineering, model training, tuning, and deployment. Cloud ML can be used for **computer vision**, **natural language processing** (**NLP**), and many other predictive analytics tasks. The Amazon cloud provides platforms for engineers and data scientists to develop ML models from end to end. In this chapter, we will discuss the following topics:

- **ML basics**: What is ML? What are the objectives of ML? What problems can be solved using ML? What are some basic ML problems?
- **Amazon SageMaker**: A fully managed AWS ML service that allows users to build, train, and deploy ML models from end to end and at scale.
- **Deep learning (DL) basics**: What are neural networks and DL models? What are their components, and how do we construct and train a DL model?
- **Amazon computer vision** solutions using DL, mainly Amazon Rekognition – a DL-based image and video analysis service.
- **Amazon NLP solutions** using DL, including the following:
  - **Amazon Comprehend**, which uncovers valuable insights and connections in text
  - **Amazon Transcribe**, which is a speech-to-text service that can transcribe audio and video recordings into text

- **Amazon Polly**, which is a text-to-speech service
- **Amazon Translate**, which translates text between languages
- **Amazon Lex**, which can be used to build application interfaces that are based on voice and text

During our discussions, we will use some examples. The sample code is available in the GitHub repository for this book: `https://github.com/PacktPublishing/Self-Taught-Cloud-computing-Engineer`.

The AWS ML services can be used individually or in combination to build a wide range of ML applications, such as recommendation systems, chatbots, and predictive systems. Let's start with an introduction to ML basics and ML pipelines.

ML basics and ML pipelines

What is ML? ML is a subfield of **artificial intelligence** (**AI**) that focuses on building models and algorithms to learn patterns and relationships from data and make predictions or decisions. A typical ML project involves the following process – the so-called ML pipeline:

- **Problem framing**: Define ML problems from business projects
- **Data collection**: Collect data from various sources, which may involve data labeling
- **Data evaluation**: Examine the data using statistical tools
- **Feature engineering**: Select and extract model features and targets
- **Model training**: Train the model with the training dataset
- **Model verification**: Verify the model with the verification dataset
- **Model testing**: Test the model with the testing dataset
- **Model deployment**: Deploy the ML model to production

Figure 6.1 shows the ML pipeline, which is an iterative process to collect data and develop ML models for deployment:

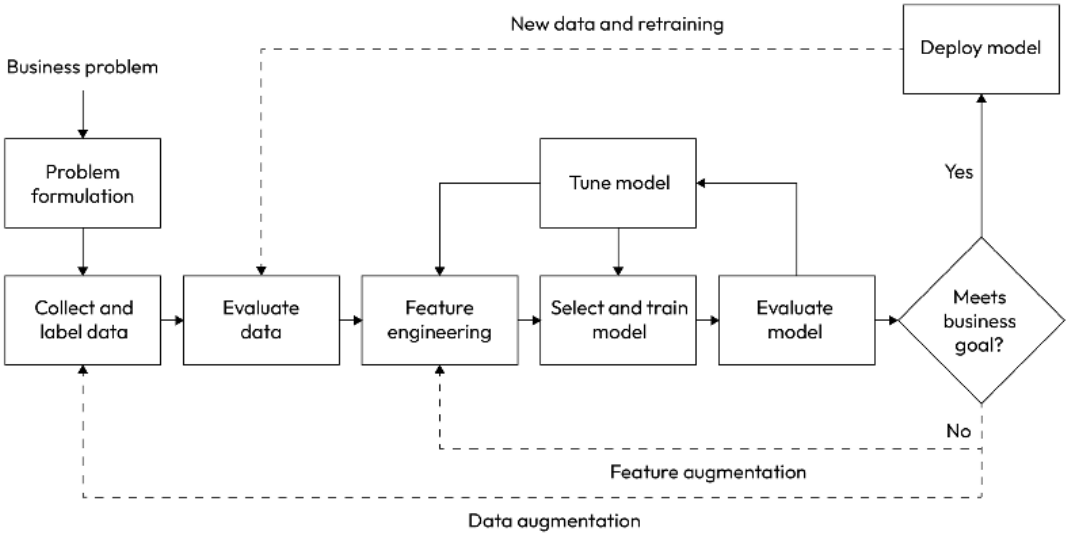


Figure 6.1 – ML pipeline

In the next subsections, we will examine the ML pipeline and discuss each stage of the pipeline in more detail.

ML problem framing

The first stage in the ML pipeline is defining the problem we need to solve and the goal we need to reach. Understanding the business goal of the problem is key in this stage since we will use the defined goal to measure the performance of our solution. During this stage, we will examine the problem and decide whether ML is the right approach to solve the problem. While ML is very useful for many business projects, it may not be the best solution for all problems. To decide whether ML is a good solution for a problem, we need to ask the following questions:

- What is the success measurement of the problem?
- What are the traditional ways of solving the problem?
- Is there enough quality data to solve the problem using ML?

After we have fully explored these questions and drawn the conclusion that ML is the best approach for the problem, we will further review the problem and data and see what type of ML problem it is. ML problems can be broadly classified into three categories:

- **Supervised learning** is training a model based on labeled data and then using the trained model to predict the output for new input data. For example, we can predict house prices based on historical sales data.

- **Unsupervised learning** involves training a model on unlabeled data to identify patterns and relationships.
- **Reinforcement learning** involves training a model based on feedback from the environment, where the algorithm learns to maximize a reward signal by taking steps toward the highest reward.

Once we have framed the ML problem and determined the ML model type, we will collect and prepare the dataset.

Data collection and preparation

The objective of this stage is to make sure that the datasets represent the real ML problem and are in the right format for ML model training. Since data usually comes from many different sources in different formats, we often use statistical techniques to sample, balance, and scale datasets and handle missing values and outliers in the datasets. Python data science libraries are powerful tools to manipulate data at this stage. This includes NumPy, Pandas, Matplotlib, and Seaborn:

- **NumPy** is a general-purpose array-processing Python library that is very good at carrying out array operations.
- **Pandas** is considered a powerful data analysis and manipulation library. It contains data structures for storing and manipulating multi-dimensional arrays.
- **Matplotlib** is a graphics Python library for data visualization. It is well integrated with NumPy and pandas.
- **Seaborn** is another data visualization library. It supports built-in Python types, such as lists and dictionaries, and objects from pandas and NumPy.

We will leverage these Python data science libraries in this chapter.

Feature engineering

After we have explored the data, we need to select and extract the model features, which are the columns of data that have the most impact on the model. **Feature selection** refers to selecting the *features* that are most relevant to the model *target*. We want to select a good number of features to avoid model underfitting or overfitting, and also build up valuable information by formatting, transforming, or combining features.

For example, **feature crossing** is a feature extraction method to synthesize two or more features into one, which will provide predictive abilities beyond what the original features can provide individually. *Figure 6.2* shows the relationship between the *target* and *features* (x and y) of a model. As we can see, it is difficult to build a model on these features, but if we synthesize a new feature from the original using a formula ($x*x + y*y$), then the model will be much easier to understand and process:

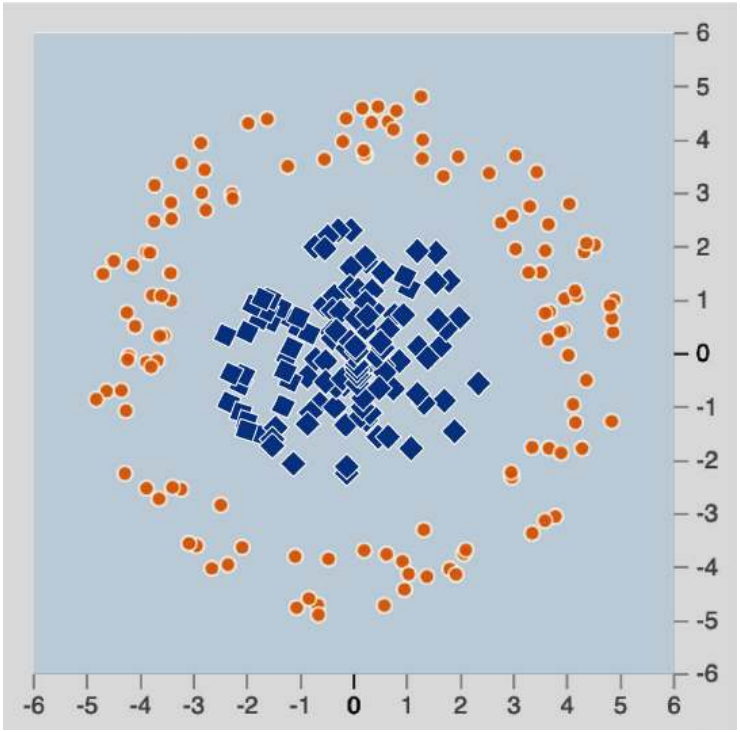


Figure 6.2 – Feature crossing

Another example is transforming categorical feature values into numerical, since computers only understand numbers. For ordinal categories such as cost, we can encode them using numbers, for example, *Low* to 1, *Medium* to 2, *High* to 3, and *Very High* to 4. But for non-ordinal categories, such as color, we need to use *one-hot encoding* to transform the different colors, as shown in *Figure 6.3*:

Color
Red
Blue
Green
Blue
Green

→

Red	Blue	Green
1	0	0
0	1	0
0	0	1
0	1	0
0	0	1

Figure 6.3 – One-hot encoding

There are also times when we need to handle missing values and outliers in the dataset. Depending on the case, we can either drop the missing values or impute them with the mean or median value for the feature. Outliers are values that lie far from other values, and we may need to delete, scale, or impute them based on the specific situation.

After we have prepared the data and extracted the features, we are ready for the next stage: model development.

ML model development

The ML model development process is a pipeline itself, including model training, model validation, and model hyperparameter tuning.

ML model training is an iterative process with the objective to minimize the *loss function* (also called the cost function), which measures the gap between the forecasted target value and the actual target value.

For regression models, we often use the **mean absolute error** (**MAE**) and **mean squared error** (**MSE**) as loss functions. `Denote N` is the number of samples. (x_i, y_i) is the coordinate of the `i`th sample data point; that is, if y_i is the actual value for x_i, and $\hat{y}$ is the predicted value for x_i, then the loss function can be defined as MAE or MSE.

MAE is the sum of the absolute differences between the predicted and the true values:

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

MSE is the sum of the squared differences between the predicted and the true values:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

For classification models, the cost function is the difference between the probability distributions for different classes. For binary classification models where the model outputs are binary (`1` for yes or `0` for no), we use **binary cross-entropy**; for multi-class classification models, we use **categorical cross-entropy** or **sparse categorical cross-entropy**, based on the input dataset labels. More details are available at `https://gombru.github.io/2018/05/23/cross_entropy_loss/`.

To minimize the loss function, we leverage many ML algorithms, such as **linear regression**, **logistic regression**, **gradient descent**, **support vector machine**, **decision trees**, and **random forest**. For more details about these algorithms and how to choose the right one, please refer to `https://learn.microsoft.com/en-us/azure/machine-learning/how-to-select-algorithms`.

Among these ML algorithms, there is a popular one that we will use in this chapter – called **Extreme Gradient Boosting** (**XGBoost**). XGBoost is a class of boosting algorithms that iteratively adds weak learners to a model to improve its performance. Building on decision trees and using a gradient-boosting framework, XGBoost starts from a very general model and then refines it by adding weak learners using gradient descent methods. Over the years, XGBoost has become a popular choice in the ML community, and it is available in several programming languages, including Python, R, Java, and Scala.

For more details, please refer to `https://www.nvidia.com/en-us/glossary/data-science/xgboost/`.

ML model validation is the stage to verify the performance of the trained model. Typically, at the beginning of an ML model development process, we divide the dataset into training, validation, and testing datasets. These datasets are randomly split and are independent of each other. In the validation process, we also use the loss function to measure the gap between the predicted and actual target values for the validation dataset, and thus validate the model performance.

As we discussed earlier, the MAE and MSE can be used to validate regression models. For classification models, a **confusion matrix** is usually created to measure the model's performance. *Figure 6.4* shows a confusion matrix example for a binary classification problem of predicting whether an image is a cat. As you can see, **true positive** (**TP**) refers to cases where the image is predicted to be a cat and it is a cat; **false positive** (**FP**) refers to when the image is predicted to be a cat but it is not in fact a cat; **false negative** (**FN**) refers to when the image is predicted to not be a cat but it is a cat; and **true negative** (**TN**) is when the image is predicted not to be a cat and it is not a cat.

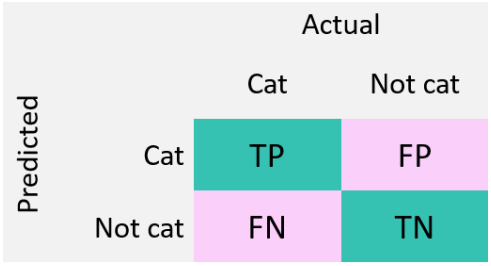


Figure 6.4 – Confusion matrix

Once we have created the confusion matrix, we can calculate the model performance stats for validation. The most used stats are as follows:

- **Recall** (**sensitivity**) measures the proportion of actual positives that were identified correctly:

  Recall = TP/(TP+FN)

- **Specificity** measures the proportion of actual negatives that were identified correctly:

  Specificity = TN/(FP+TN)

- **Precision** measures the proportion of positive identifications that are correct:

  Precision=TP/(TP+FP)

More details on model validation and the confusion matrix can be found at `https://www.sciencedirect.com/topics/engineering/confusion-matrix` and `https://towardsdatascience.com/supervised-machine-learning-model-validation-a-step-by-step-approach-771109ae0253`.

Many times during model training and validation, we come across model underfitting and overfitting. **Model underfitting** describes the situation where the prediction error is not minimized, and model overfitting is when the model fits the training dataset very well but does not fit the validation dataset. We want to avoid both underfitting and overfitting and make a good fit/robust model. There are many techniques we can use.

For more details, please refer to `https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/`.

ML model hyperparameter tuning is the process of selecting the best set of hyperparameters for an ML model to maximize its performance on a given dataset. Hyperparameters are not learned by the model during training but are specified by the user at the beginning of the training process, such as the learning rate, number of epochs, and batch size.

For more details, please refer to `https://cloud.google.com/ai-platform/training/docs/hyperparameter-tuning-overview`.

ML model deployment, testing, and monitoring

After the model is trained, tuned, and validated, we will deploy it in a production environment, with the necessary hardware and software to host the prediction service. Model testing means testing a model in a production-like environment and monitoring its performance to detect any issues. After the model is deployed to the production environment and tested, we still need to monitor the model performance and retrain it as needed.

Now that we have introduced the basic ML concepts and ML pipeline, let's take a look at Amazon SageMaker and go through the ML pipeline process under SageMaker.

Amazon SageMaker

Amazon SageMaker provides a fully managed cloud platform for users to develop ML models from end to end. Some of the key features of Amazon SageMaker are as follows:

- **Data preparation**: Amazon SageMaker provides various tools to preprocess and prepare data
- **Model training algorithms**: SageMaker provides built-in algorithms for supervised learning, unsupervised learning, and reinforcement learning
- **Model deployment**: After the ML model is trained and validated, SageMaker provides tools for model deployment, either as a batch transform job or a real-time endpoint
- **Scalability**: SageMaker is a fully managed service, which means that AWS takes care of all the infrastructure and scaling, so the data scientists can focus on building better models rather than worrying about infrastructure

- **Integration**: SageMaker integrates with other AWS services, such as S3, AWS Glue, and AWS Lambda, so data scientists can easily access and use datasets stored in AWS

Next, we will use Amazon SageMaker to solve a real-life problem. The details of the problem are as follows:

- **Problem definition**: A healthcare provider wants to improve the success rate of detecting abnormalities in orthopedic patients. The problem will be solved using ML. The dataset contains six biomechanical features and a target of normal or abnormal. We will use it to train an ML model and predict whether a patient will have an abnormality.
- **The dataset**: We will use the public vertebral column dataset from the UCI Machine Learning Repository (`https://archive.ics.uci.edu/ml/datasets/vertebral%2Bcolumn`).
- **The platform**: We will use Amazon SageMaker to collect and prepare the datasets, conduct model training and validation, deploy the model and perform batch transformation for the testing dataset, and fine-tune the model.

Are you ready? Let's get our hands dirty to develop an ML pipeline using Amazon SageMaker:

1. **Launch a Jupyter notebook**:

Log in to the AWS Management Console. From the **Services** menu, choose **Amazon SageMaker**. In the navigation menu on the left, expand the **Notebook** section and choose **Notebook instances**. Click **Create notebook instance**.

In the **Notebook instance name** box, enter a name for the notebook, such as `notebook1`. From the **Notebook instance type** drop-down list, choose **ml.t3.medium**. Set **Platform identifier type** to the latest one (**notebook-al2-v2**). Leave the remaining settings at their default values. Click **Create notebook instance**, as shown in *Figure 6.5*:

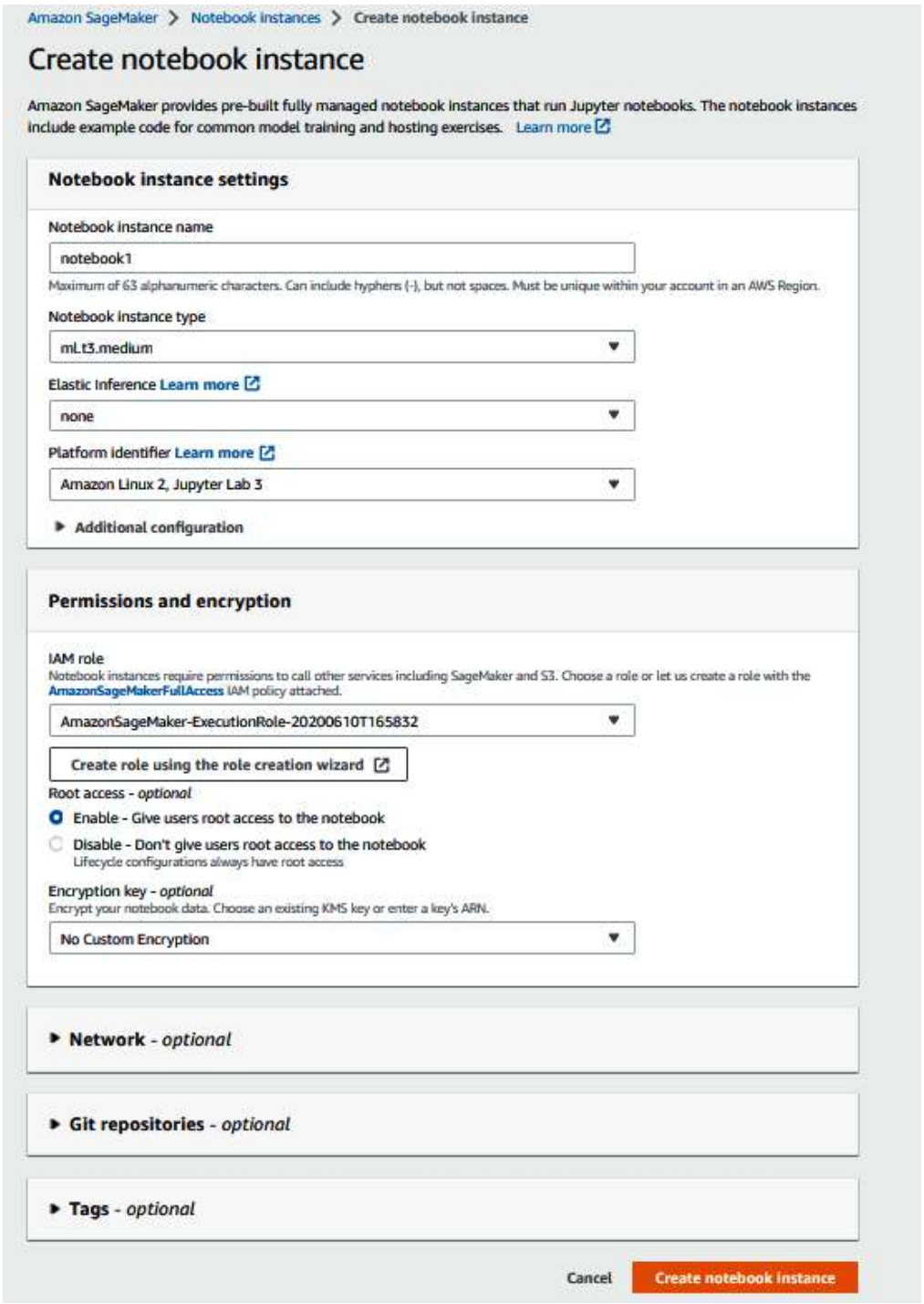


Figure 6.5 – Launch a SageMaker notebook instance

When it's initially created, the new notebook instance will show as **Pending**. After it changes to **InService**, choose **Open JupyterLab** at the end of the row. A new window opens with the launcher. Choose **conda_python3**.

2. **Working in Jupyter Notebook**

   Now, the Jupyter notebook is ready to execute the ML pipeline process, using Python code. We will develop the pipeline step by step:

   I. **Import the Python data science libraries**:

```
import warnings, requests, zipfile, io
warnings.simplefilter('ignore')
import pandas as pd
from scipy.io import arff
import boto3
```

   II. **Import the dataset and load data into a pandas DataFrame**:

```
f_zip=
'http://archive.ics.uci.edu/ml/machine-learning-databases/00212/
vertebral_column_data.zip'
r=requests.get(f_zip, stream=True)
Vertebral_zip=zipfile.ZipFile(io.BytesIO(r.content))
Vertebral_zip.extractall()
```

The data is imported and ready for use, and you will see four files show up on the left panel, as shown in *Figure 6.6*. We can load them into pandas DataFrames:

```
data = arff.loadarff('column_2C_weka.arff')
df = pd.DataFrame(data[0])
class_mapper = {b'Abnormal':1,b'Normal':0}
df['class']=df['class'].replace(class_mapper)
```

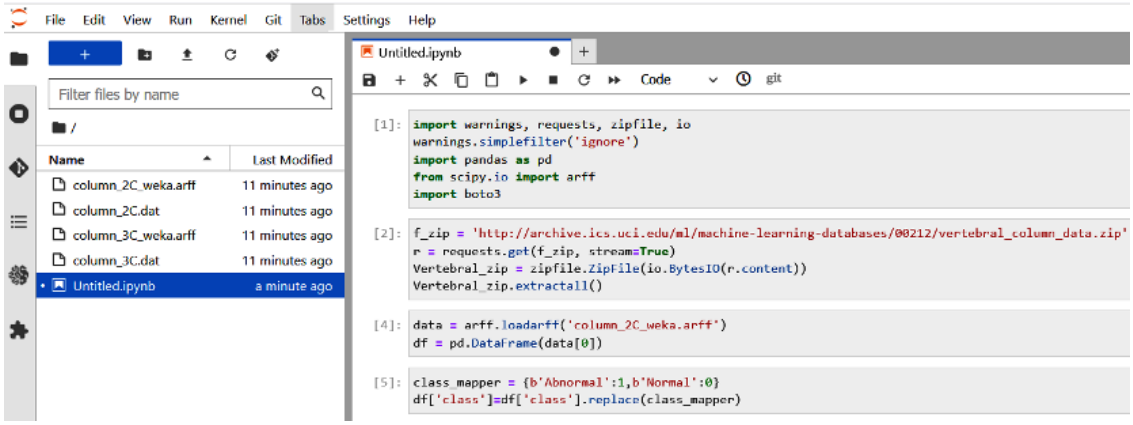


Figure 6.6 – Import the dataset and load data into DataFrames

III. **Explore the data**

Use `df.shape` to examine the number of rows and columns and `df.columns` to list the six biomechanical features. The target column is named `class`.

IV. **Prepare the data**

Since XGBoost requires the training data to be in a single file with the first column as the target value, we need to move the target column to the first position. Then, we split the dataset into training, validate, and test datasets and examine the three datasets.

Figure 6.7 shows the execution results of steps III and IV:

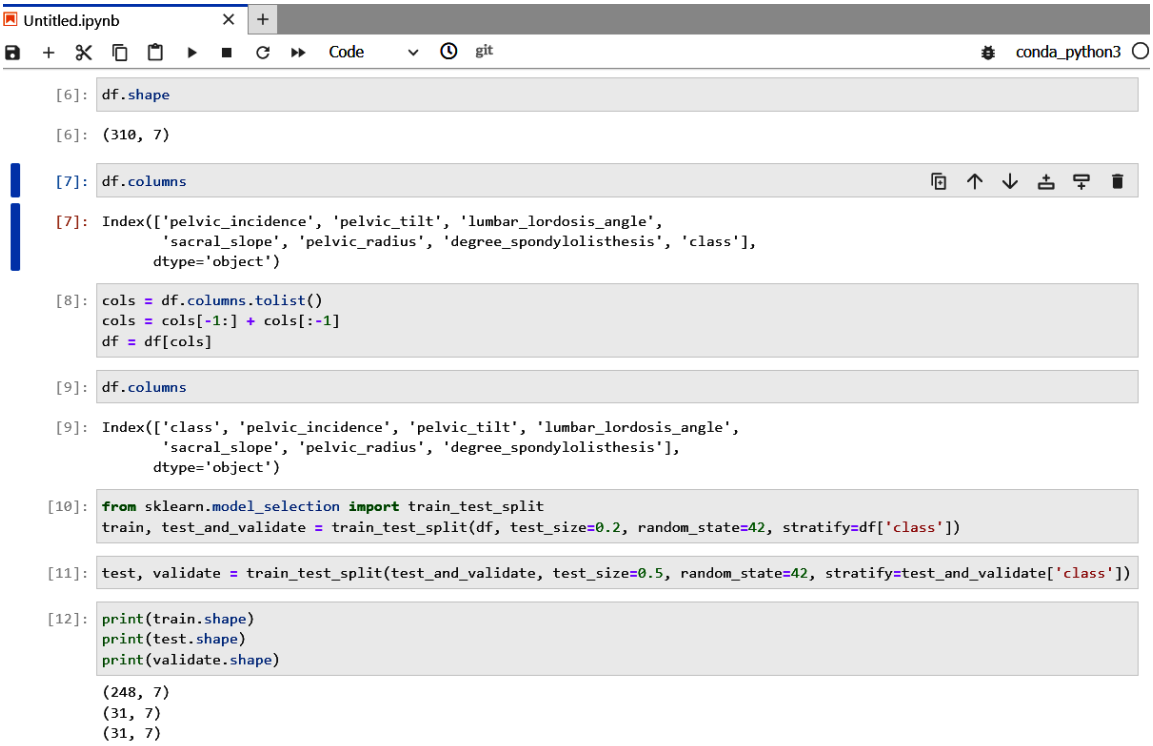


Figure 6.7 – Explore and prepare data

V. **Upload the data to Amazon S3**

Since XGBoost loads the training data from S3, we need to create an S3 bucket, write the data to a **comma-separated values** (**CSV**) file, and then upload the file to Amazon S3:

```
bucket='arn:aws:s3:::juyper03172023'
prefix='lab'
train_file='vertebral_train.csv'
test_file='vertebral_test.csv'
validate_file='vertebral_validate.csv'
```

```
import os
s3_resource = boto3.Session().resource('s3')
def upload_s3_csv(filename, folder, dataframe):
    csv_buffer = io.StringIO()
    dataframe.to_csv(csv_buffer, header=False, index=False)
    s3_resource.Bucket(bucket).Object(os.path.join(prefix,
folder, filename)).put(Body=csv_buffer.getvalue())
```

Figure 6.8 shows the execution results of step V:

```
[23]: bucket='juyper03172023'
prefix='lab'

train_file='vertebral_train.csv'
test_file='vertebral_test.csv'
validate_file='vertebral_validate.csv'

[24]: import os
s3_resource = boto3.Session().resource('s3')

[25]: def upload_s3_csv(filename, folder, dataframe):
    csv_buffer = io.StringIO()
    dataframe.to_csv(csv_buffer, header=False, index=False)
    s3_resource.Bucket(bucket).Object(os.path.join(prefix, folder, filename)).put(Body=csv_buffer.getvalue())

[26]: upload_s3_csv(train_file, 'train', train)
upload_s3_csv(test_file, 'test', test)
upload_s3_csv(validate_file, 'validate', validate)
```

Figure 6.8 – Upload data to S3

Now we are ready to train the model!

VI. **Train the model using XGBoost**

The first step is to get the XGBoost container URL:

```
import boto3
from sagemaker.image_uris import retrieve
container = retrieve('xgboost',boto3.Session().region_name,'1.0-
1')
```

Set initial hyperparameters for the model and use the estimator function to set the model up:

```
hyperparams={"num_round":"42",
             "eval_metric": "auc",
             "objective": "binary:logistic"}

import sagemaker
s3_output_location="s3://{}/{}/output/".format(bucket,prefix)
xgb_model=sagemaker.estimator.Estimator(container,
                                        sagemaker.get_execution_
role(),
instance_count=1,
```

```
                                                    instance_type='ml
.m4.xlarge',
                                                    output_path=s3_
output_location,
                                                     hyperparameters=
hyperparams,
                                                     sagemaker_session=
sagemaker.Session())
```

The estimator needs **channels** to feed data into the model. We will use `train_channel` and `validate_channel`:

```
train_channel = sagemaker.inputs.TrainingInput(
    "s3://{}/{}/train/".format(bucket,prefix,train_file),
    content_type='text/csv')

validate_channel = sagemaker.inputs.TrainingInput(
    "s3://{}/{}/validate/".format(bucket,prefix,validate_file),
    content_type='text/csv')

data_channels = {'train': train_channel, 'validation':
validate_channel}
```

Figure 6.9 shows the execution in the notebook of these steps.

[27]:
```
import boto3
from sagemaker.image_uris import retrieve
container = retrieve('xgboost',boto3.Session().region_name,'1.0-1')
```

[28]:
```
hyperparams={"num_round":"42",
             "eval_metric": "auc",
             "objective": "binary:logistic"}
```

[29]:
```
import sagemaker
s3_output_location="s3://{}/{}/output/".format(bucket,prefix)
xgb_model=sagemaker.estimator.Estimator(container,
                                        sagemaker.get_execution_role(),
                                        instance_count=1,
                                        instance_type='ml.m4.xlarge',
                                        output_path=s3_output_location,
                                         hyperparameters=hyperparams,
                                         sagemaker_session=sagemaker.Session())
```

[30]:
```
train_channel = sagemaker.inputs.TrainingInput(
    "s3://{}/{}/train/".format(bucket,prefix,train_file),
    content_type='text/csv')

validate_channel = sagemaker.inputs.TrainingInput(
    "s3://{}/{}/validate/".format(bucket,prefix,validate_file),
    content_type='text/csv')

data_channels = {'train': train_channel, 'validation': validate_channel}
```

Figure 6.9 – Split dataset and define estimator

Now, train the model:

```
xgb_model.fit(inputs=data_channels, logs=False)
```

Figure 6.10 shows a screenshot of the model training:

```
[31]: xgb_model.fit(inputs=data_channels, logs=False)

INFO:sagemaker:Creating training-job with name: sagemaker-xgboost-2023-03-17-22-16-09-934

2023-03-17 22:16:10 Starting - Starting the training job.......
2023-03-17 22:16:50 Starting - Preparing the instances for training.............
2023-03-17 22:17:53 Downloading - Downloading input data.....
2023-03-17 22:18:23 Training - Downloading the training image........
2023-03-17 22:19:09 Training - Training image download completed. Training in progress....
2023-03-17 22:19:29 Uploading - Uploading generated training model..
2023-03-17 22:19:45 Completed - Training job completed
```

Figure 6.10 – Model training

We can see that the model training has completed.

VII. **Conduct a batch transform on the testing dataset**

We will now construct a batch transformation, as seen in *Figure 6.11*.

```
batch_X = test.iloc[:,1:];

batch_X_file='batch-in.csv'
upload_s3_csv(batch_X_file, 'batch-in', batch_X)

batch_output = "s3://{}/{}/batch-out/".format(bucket,prefix)
batch_input = "s3://{}/{}/batch-in/{}".format(bucket,prefix,batch_X_file)

xgb_transformer = xgb_model.transformer(instance_count=1,
                                        instance_type='ml.m4.xlarge',
                                        strategy='MultiRecord',
                                        assemble_with='Line',
                                        output_path=batch_output)

xgb_transformer.transform(data=batch_input,
                          data_type='S3Prefix',
                          content_type='text/csv',
                          split_type='Line')
xgb_transformer.wait()

s3 = boto3.client('s3')
obj = s3.get_object(Bucket=bucket, Key="{}/batch-out/{}".format(prefix,'batch-in.csv.out'))
target_predicted = pd.read_csv(io.BytesIO(obj['Body'].read()),',',names=['class'])
```

```
INFO:sagemaker:Creating model with name: sagemaker-xgboost-2023-03-18-04-02-54-790

INFO:sagemaker:Creating transform job with name: sagemaker-xgboost-2023-03-18-04-02-55-529

2023-03-18T04:08:52.134:[sagemaker logs]: MaxConcurrentTransforms=4, MaxPayloadInMB=6, BatchStrategy=MULTI_RECORD
```

Figure 6.11 – Batch transformation

After the batch transformation step, we can build a confusion matrix.

VIII. **Create a confusion matrix**

Build a function to convert the positive probability into binary (`0` or `1`):

```
def binary_convert(x):
    threshold = 0.3
    if x > threshold:
        return 1
    else:
        return 0

target_predicted_binary = target_predicted['class'].
apply(binary_convert)

print(target_predicted_binary.head(5))
test.head(5)
```

Figure 6.12 shows the execution in a notebook.

```
[5]: def binary_convert(x):
         threshold = 0.3
         if x > threshold:
             return 1
         else:
             return 0

     target_predicted_binary = target_predicted['class'].apply(binary_convert)

     print(target_predicted_binary.head(5))
     test.head(5)
```

```
0    1
1    1
2    1
3    1
4    1
Name: class, dtype: int64
```

[5]:

	class	pelvic_incidence	pelvic_tilt	lumbar_lordosis_angle	sacral_slope	pelvic_radius	degree_spondylolisthesis
136	1	88.024499	39.844669	81.774473	48.179830	116.601538	56.766083
230	0	65.611802	23.137919	62.582179	42.473883	124.128001	-4.083298
134	1	52.204693	17.212673	78.094969	34.992020	136.972517	54.939134
130	1	50.066786	9.120340	32.168463	40.946446	99.712453	26.766697
47	1	41.352504	16.577364	30.706191	24.775141	113.266675	-4.497958

Figure 6.12 – Forecast testing dataset

Then, create a confusion matrix using the target values from the test dataset and the predicted value:

```
test_labels = test.iloc[:,0]
test_labels.head()
from sklearn.metrics import confusion_matrix
matrix = confusion_matrix(test_labels, target_predicted_binary)
df_confusion = pd.DataFrame(matrix,
index=['Normal','Abnormal'],columns=['Normal','Abnormal'])
df_confusion
```

Figure 6.13 shows the execution in a notebook:

```
[6]: test_labels = test.iloc[:,0]
     test_labels.head()

[6]: 136    1
     230    0
     134    1
     130    1
     47     1
     Name: class, dtype: int64
```

Now, you can use the *scikit-learn* library, which contains a function to create a confusion matrix.

```
[7]:
     from sklearn.metrics import confusion_matrix

     matrix = confusion_matrix(test_labels, target_predicted_binary)
     df_confusion = pd.DataFrame(matrix, index=['Nnormal','Abnormal'],columns=['Normal','Abnormal'])

     df_confusion
```

[7]:

	Normal	Abnormal
Nnormal	7	3
Abnormal	2	19

Figure 6.13 – Create a confusion matrix

Now, we can calculate the performance stats for the model.

IX. **Calculate the performance stats**

Now calculate the stats:

```
Sensitivity  = float(TP)/(TP+FN)*100
print(f"Sensitivity or TPR: {Sensitivity}%")
Specificity  = float(TN)/(TN+FP)*100
print(f"Specificity or TNR: {Specificity}%")
```

```
Precision = float(TP)/(TP+FP)*100
print(f"Precision: {Precision}%")
```

Figure 6.14 shows the execution in a notebook.

```
[21]: Sensitivity  = float(TP)/(TP+FN)*100
      print(f"Sensitivity or TPR: {Sensitivity}%")
      Specificity  = float(TN)/(TN+FP)*100
      print(f"Specificity or TNR: {Specificity}%")
      Precision = float(TP)/(TP+FP)*100
      print(f"Precision: {Precision}%")

      Sensitivity or TPR: 90.47619047619048%
      Specificity or TNR: 70.0%
      Precision: 86.36363636363636%
```

Figure 6.14 – Performance stats

In this section, we introduced Amazon SageMaker and showed the ML pipeline, from data collection and preparation, to model training and validation, to model deployment, testing, and fine-tuning, using SageMaker. In the next sections, we will introduce DL and explore the Amazon computer vision and NLP solutions using DL.

DL basics

DL was introduced in 2012. The basic idea is to mimic the human brain and construct **artificial neural networks** (**ANNs**) to train models. A typical multi-layer ANN has three types of layers: an **input layer**, one or more **hidden layers**, and an **output layer**. *Figure 6.15* shows an ANN that has one input layer, two hidden layers, and an output layer. In the ANN, a circular node represents a perceptron, and a line represents the connection from the output of one perceptron to the input of another.

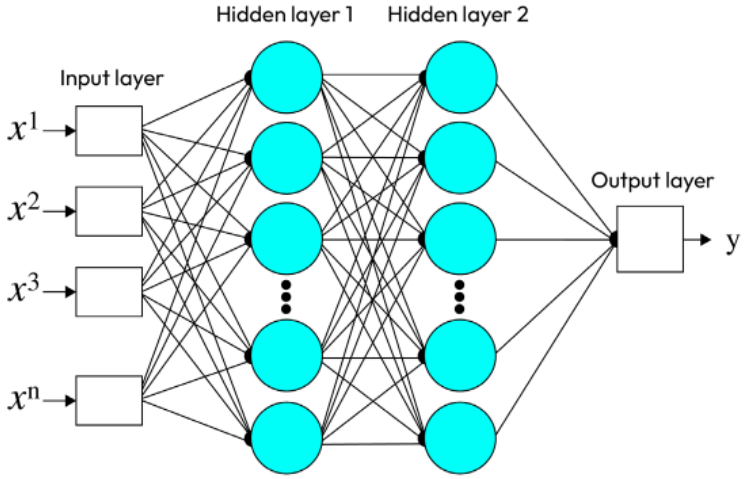


Figure 6.15 – A multi-layer ANN

The objective of DL model training is the same as ML: minimize the loss function, which is defined as the gap between the model's predicted value and the actual value. Different from traditional ML algorithms, DL uses the activation function to add nonlinearity to the model training process.

In a typical DL model, we define the following to construct a neural network:

- The **layers of the model** (input layer, hidden layers, and output layers)
- The **activation function** for each layer (such as ReLU or softmax)
- The **optimizer**, which is the DL algorithm used to train the model
- The **loss function** (such as MAE, MSE, and `categorical_crossentropy`)
- The **dropout rate**, which is the percentage of nodes and their incoming/outgoing connections to be temporarily removed from the network – to avoid model overfitting

The following code snippet shows an example of building and training a DL model using **Keras**, which is a software library for constructing DL models. As we can see, it builds a neural network of three layers: the input layer with 16 nodes, 2 hidden layers with 128 and 24 nodes (`0.25` dropout), and an output layer of 3 nodes (softmax activation). The optimizer algorithm is **stochastic gradient descent** (**SGD**), and the loss function is `categorical_crossentropy`, which fits multi-classification models:

```
#Import keras
model = keras.sequential
model.add(layers.Dense(128,activation='relu',input_shape=(16,
)))
model.add(layers.Dense(24,activation='relu'))
model.add(layers.Dropout(rate=0.25))
model.add(layers.Dense(3,activation='softmax'))
```

```
model.compile(sgd(lr=0.5), loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(train_set, train_label, batch_size=20, epochs=30,
shuffle=True)
```

Since DL was introduced, it has made breakthroughs in a lot of AI areas, including computer vision and NLP. Many applications are developed using pretrained neural DL models. In the next section, we will introduce the Amazon DL solutions in computer vision and NLP.

Amazon computer vision solutions

Computer vision is the automated extraction of information from digital images. To solve computer vision problems, Amazon provides **Rekognition** – a fully managed service using DL to analyze images and videos. Here are some key features of Amazon Rekognition:

- **Object and scene detection**: Rekognition can identify objects and scenes within an image or video frame, including vehicles, buildings, animals, and landscapes.
- **Facial analysis and facial comparison**: Rekognition can detect faces in images and videos and perform facial analysis, including facial recognition, gender identification, age estimation, and facial expression analysis. It can compare and match faces from different images.
- **Text detection**: Rekognition can detect and extract text in images and videos, including printed and handwritten text.
- **Custom labels**: Rekognition also allows you to create custom labels for specific objects or scenes that are important to your business or use case.
- **Video analysis**: Rekognition can analyze videos and extract specific frames or segments based on your criteria, such as facial recognition or object detection.

Amazon Rekognition provides a powerful set of image and video analysis capabilities that can be used in a variety of applications, such as security, media analysis, customer engagement, and content moderation. Next, we will show two of its functions: label detection and facial recognition and matching.

Label detection

In the AWS Management Console, select **Amazon Rekognition** and go to the **Rekognition** console. Select **Demos | Label detection**. When you upload an image, it analyzes the content and returns a list of labels that describe the visual elements of the content. *Figure 6.16* shows the label detection results:

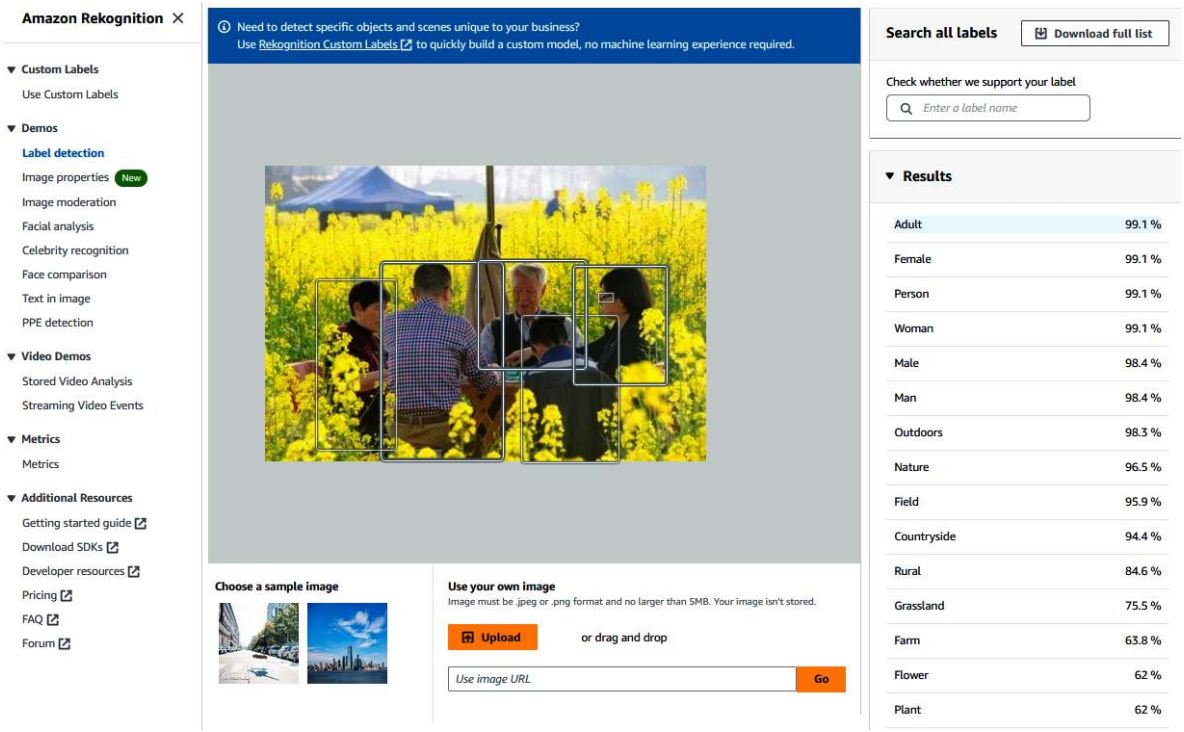


Figure 6.16 – Label detection

Facial recognition and matching

In the AWS Management Console, select **Amazon Rekognition** and go to the **Rekognition** console. Select **Demos | Face comparison**. Upload an image as a reference face and another image for the comparison faces. Rekognition compares the reference face with each of the comparison faces and returns the results. *Figure 6.17* shows the face comparison results:

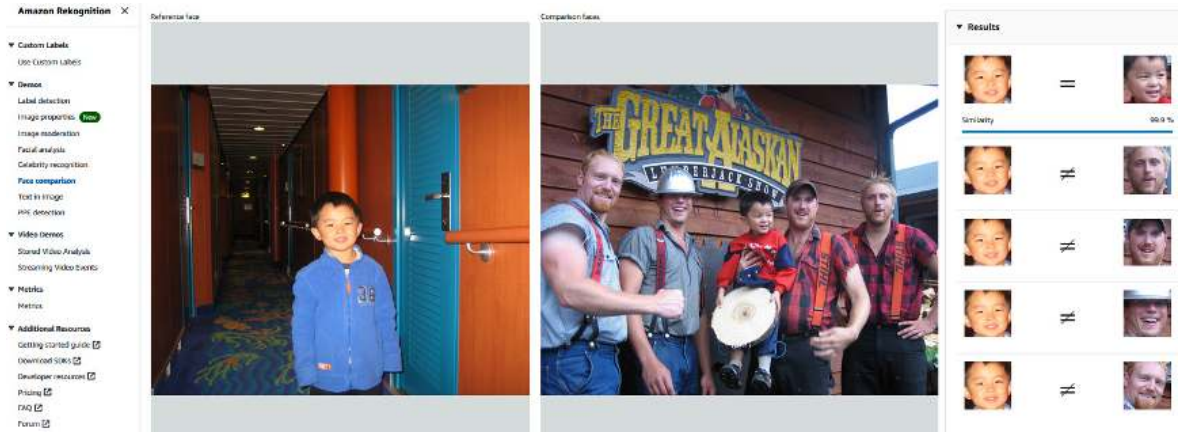


Figure 6.17 – Face comparison

Due to space limits, we have only shown some Amazon Rekognition features. Please feel free to explore more computer vision solutions and have fun!

In the next section, we will look at Amazon's NLP services.

Amazon's NLP solutions

Amazon offers a range of NLP solutions:

- **Amazon Comprehend**, which is a service to perform sentiment analysis, entity recognition, and topic modeling on large volumes of unstructured text
- **Amazon Transcribe**, which converts speech to text and supports multiple languages and a variety of audio formats
- **Amazon Polly**, which is a text-to-speech service
- **Amazon Translate**, which provides high-quality translations for a variety of languages
- **Amazon Lex**, which is a service for building chatbots and conversational interfaces

Amazon Comprehend

Amazon Comprehend uses DL algorithms to analyze and understand text data in multiple languages. Amazon Comprehend can perform the following functions:

- **Language detection**: Automatically detect the language of the text data, which is useful in multilingual applications
- **Sentiment analysis**: Determine the sentiment of text data: positive, negative, or neutral
- **Entity recognition**: Identify and extract entities from text data, such as people, organizations, locations, and dates
- **Key phrase extraction**: Extract key phrases from text data, such as important topics or keywords
- **Topic modeling**: Identify topics in text data and group them based on their similarity
- **Custom classification**: Allows users to create custom classifiers for specific use cases, such as categorizing customer feedback into different categories

Amazon Comprehend provides a powerful set of NLP capabilities that can be used in a variety of applications, such as customer service, social media monitoring, content analysis, and compliance monitoring.

Amazon Transcribe

Amazon Transcribe is a speech recognition service that uses advanced ML algorithms to convert audio and video files into text transcripts in real time or asynchronously. Here are some key features of Amazon Transcribe:

- **Speech-to-text conversion**: It can transcribe audio and video files into text transcripts, with support for multiple languages and dialects
- **Real-time transcription**: It can transcribe live audio and video streams in real time, making it useful for applications such as call centers and live events
- **Speaker identification**: It can identify individual speakers in a multi-speaker audio or video file and label each spoken word with the corresponding speaker ID
- **Automatic punctuation**: It can add appropriate punctuation to the transcribed text, making it easier to read and understand
- **Custom vocabulary**: It allows you to upload custom vocabularies that can be used to improve the accuracy of the transcription for a domain-specific language
- **Compliance and security**: It is designed with strong security and compliance features, such as encryption, access controls, and compliance with GDPR and other regulations

Amazon Transcribe provides powerful speech-to-text capabilities that can be used in a variety of applications, such as call transcription, captioning, subtitling, and voice search.

Amazon Polly

Amazon Polly is a text-to-speech service that uses advanced DL technologies to create natural-sounding speech from text. Amazon Polly supports a wide range of languages and voices, including male and female voices with different accents and styles. The service can generate speech in real time, allowing applications to provide dynamic and responsive voice output. Amazon Polly also supports the use of **Speech Synthesis Markup Language** (**SSML**), which allows developers to fine-tune the pronunciation and intonation of the generated speech. Some of the key benefits of using Amazon Polly include the following:

- **High-quality speech output**: Amazon Polly uses advanced neural text-to-speech technology to create natural-sounding speech with lifelike intonation and pronunciation.
- **Customizability**: Developers can control various aspects of the speech output, such as voice, intonation, and pronunciation, to create a more personalized and engaging experience.
- **Cost-effectiveness**: Amazon Polly offers pay-as-you-go pricing, which means that you only pay for what you use.
- **Easy integration**: Amazon Polly provides a simple API that developers can use to integrate speech capabilities into their applications. The service can be used with a variety of programming languages and platforms, including Java, Python, and .NET.

Amazon Polly can be used in e-learning, gaming, accessibility, and more. It can also be used in conjunction with other AWS services, such as Amazon Lex and Amazon Comprehend, to create more sophisticated and intelligent voice-based applications.

Amazon Translate

Amazon Translate is a fully managed machine translation service that translates text in multiple languages. Some key features of Amazon Translate are as follows:

- **Language translation**: Translate can translate text from one language to another, with support for over 70 languages
- **Real-time translation**: Translate can translate text in real time, making it useful for applications such as chatbots and customer support
- **Batch translation**: Translate can translate large volumes of text in batches, making it useful for tasks such as website localization
- **Custom terminology**: Translate allows you to upload custom terminology that can be used to improve the accuracy of the translation for a domain-specific language
- **Neural machine translation**: Translate uses a neural machine translation engine that provides high-quality translations that are more accurate and natural sounding than traditional machine translation methods
- **Automatic language detection**: Translate can automatically detect the language of the input text, making it easier to use in multilingual applications
- **Compliance and security**: Translate is designed with strong security and compliance features, such as encryption, access controls, and compliance with GDPR and other regulations

Amazon Translate provides a powerful set of machine translation capabilities that can be used in a variety of applications, such as e-commerce, customer service, and content localization. It allows businesses to expand their reach globally by providing quick and accurate translations of their content in multiple languages.

Amazon Lex

Amazon Lex is a service that enables developers to build conversational interfaces and chatbots using NLP and **automatic speech recognition** (**ASR**) technologies. Amazon Lex can be used to create chatbots for a wide range of applications, including customer service, e-commerce, and education. Some of the key features of Amazon Lex include the following:

- **Natural language understanding**: Amazon Lex uses advanced NLP technology to understand natural language input and map it to intents and slots

- **Automatic speech recognition**: Amazon Lex can process spoken language and convert it into text, enabling users to interact with chatbots through voice commands
- **Multi-platform support**: Amazon Lex provides SDKs for several popular platforms, including iOS, Android, and JavaScript, enabling developers to build chatbots for web and mobile applications
- **Integration with other AWS services**: Amazon Lex can be integrated with other AWS services, such as Amazon Lambda, Amazon DynamoDB, and Amazon S3, to create more sophisticated chatbots
- **Scalability**: Amazon Lex is designed to handle large volumes of requests and can scale automatically based on demand

Using Amazon Lex, developers can create chatbots that can handle simple and complex interactions with users, such as answering questions, providing recommendations, and processing transactions. Chatbots built with Amazon Lex can be deployed across multiple channels, including web, mobile, and messaging platforms, providing a seamless and consistent user experience.

These AWS NLP services are easy to access and set up from the console or CLI. We will not show any hands-on examples here. Please feel free to explore these services by yourself.

Summary

In this chapter, we have discussed AWS ML services. We started by introducing ML concepts and the ML pipeline, then dove into Amazon SageMaker, which provides fully managed end-to-end ML services. We then introduce DL concepts and examined the AWS computer vision and NLP solutions using DL pretrained models.

So far, we have explored AWS cloud services such as compute, storage, networking, databases, big data, and ML. In the next chapter, we will discuss another important cloud topic: Amazon cloud security.

Practice questions

Questions 1-4 are based on the following use case.

ML case #1

An engineer is training an Amazon SageMaker model to detect as many true **malignant tumors** (**MTs**) from MRI images as possible. The model features are shown in *Figure 6.18*.

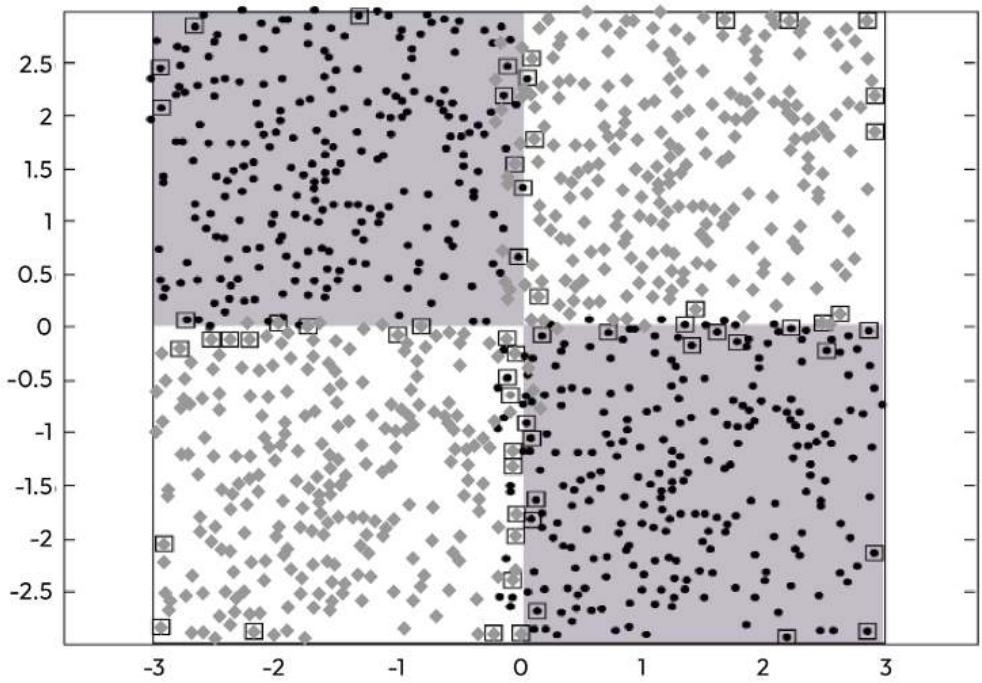


Figure 6.18 – Model features: x and y

The initial models were underfitting, so they put in a lot of effort and finally got two models working. Their confusion matrixes are shown in *Figure 6.19*:

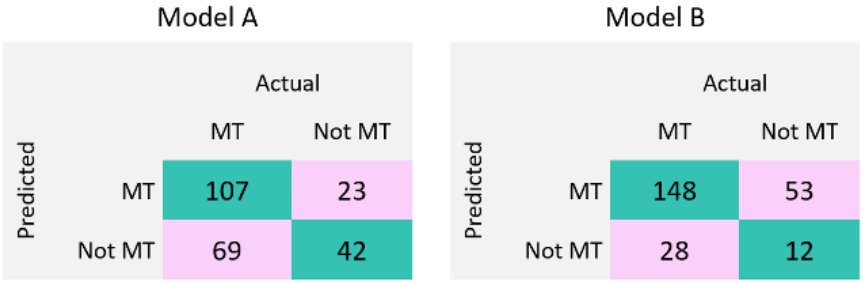


Figure 6.19 – Confusion matrixes for models A and B

1. How should they synthesize the two features, *x* and *y*?

A. *x*x + y*y*

B. *x+y*

C. *x*y*

D. *x*10 + y*10*

2. What is the precision for model B?

A. 74%

B. 84%

C. 18%

D. 50%

3. What may have helped them improve the initial model?

A. Add more features to the model

B. Add L1 regularization

C. Add L2 regularization

D. Increase the learning rate

4. Which of the following statements is true?

A. Model B is better

B. Model A is better

C. We need an ROC curve to decide

D. We need an F1 core to decide

Questions 5-10 are based on the following use case.

ML case #2

An engineer is building a DL model to predict whether images contain a driver's license, passport, or credit card. The code and initial training results are shown in *Figure 6.20*:

```
#Import keras
model = keras.sequential
model.add(layers.Dense(128,activation='relu',input_shape=(16,
)))
model.add(layers.Dense(24,activation='relu'))
model.add(layers.Dropout(rate=0.25))
model.add(layers.Dense(3,activation='softmax'))
model.compile(sgd(lr=0.5), loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(train_set, train_label, batch_size=20, epochs=30,
shuffle=True)
```

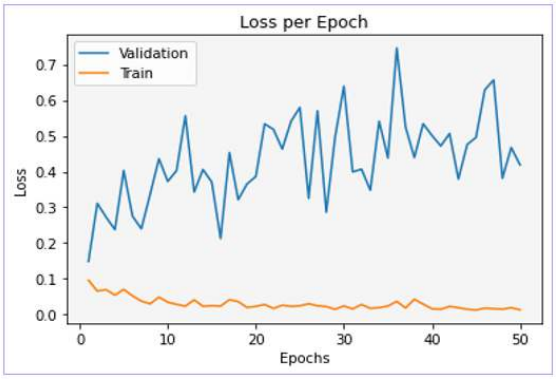


Figure 6.20 – DL model and initial training results

5. What optimizer did they use?

    a. Sequential

    b. ReLU

    C. Softmax

    D. Stochastic gradient descent

6: What kind of problem are they likely to solve?

    A. Binary classification

    B. Regression

    C. Multi-classification of three: a dog, a cat, or a giraffe

    D. Multi-classification of "a combination of three": a dog, a cat, and a giraffe

7: How many trainable weights does the model have?

    A. 16x128+128x24x0.25+24x3

    B. 16x128x24x3

    C. 16+128+24x0.25+3

    D. 16x128+128x24+24x3

8: How can they improve the model generality?

    A. Decrease the learning rate to 0.2

    B. Apply regularization

    C. Increase epochs to 60

    D. Decrease the batch size to 10

9. How can they make the model converge?

    A. Decrease the learning rate to 0.2

    B. Apply regularization

    C. Increase epochs to 60

    D. Decrease the batch size to 10

10. Which of the following is likely a label for the training dataset?

    A. `drivers_license`

    B. `passport`

    C. `credit_card`

    D. `[drivers_license, passport, credit_card]`

Questions 11-14 are based on the use case and diagrams shown in *Figure 6.21*:

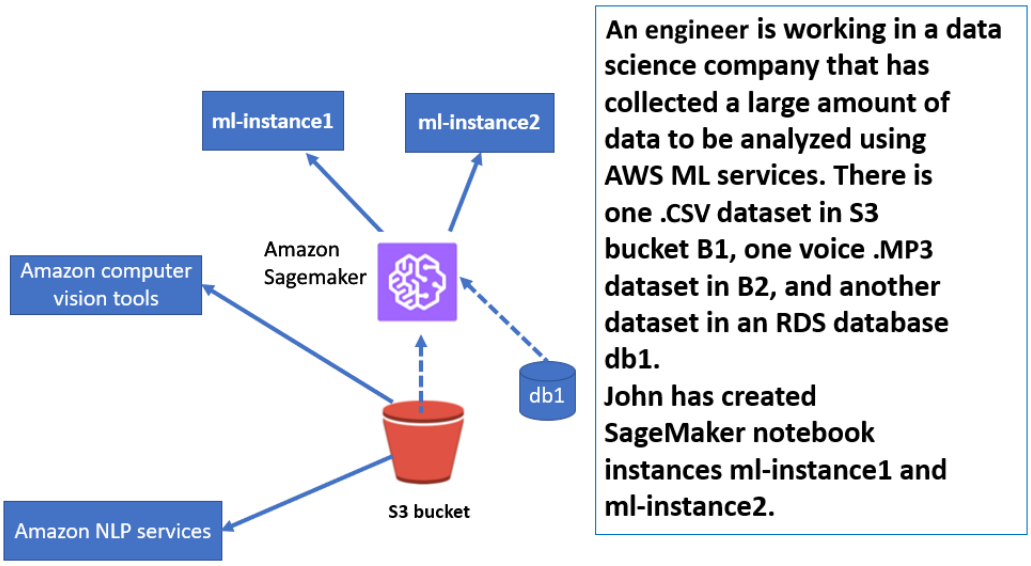


Figure 6.21 – DL model training architecture

11. They are using `ml-instance1` to train ML models using `.csv` files in B1. It has taken a long time and they want to improve the performance. What is your suggestion?

A. Use SageMaker Pipe mode

B. Copy the `.csv` files to `ml-instance1`

C. Use Amazon Kinesis to stream data from B1 to `ml-instance1`

D. Use AWS Glue to transform the CSV dataset to JSON format first

12. They want to analyze the voice recordings in B2 to understand the topics with minimum effort. What is your suggestion?

A. Amazon Transcribe + a custom NLP algorithm with Amazon SageMaker

B. SageMaker BlazingText algorithm + Amazon Transcribe

C. A custom NLP algorithm with Amazon SageMaker + Amazon Transcribe

D. Amazon Transcribe + Amazon Comprehend

13. They and their coworkers have deployed many SageMaker notebook instances, and each team member only needs access to their personal instances. What is your recommendation?

A. Create an IAM policy for each member and grant access to their personal instances only

B. Use port forwarding to prevent all internet traffic from being forwarded to the notebook instances

C. Use Amazon CloudWatch to monitor and restrict unauthorized access

D. Attach an Amazon S3 bucket policy to restrict access to the buckets that contain other users' notebook instances

14. They checked all the VPCs and could not find `ml-instance1`. What is the reason?

A. `ml-instance1` is hidden in the VPCs of John's account

B. `ml-instance1` is run in the ECS service of John's account

C. `ml-instance1` is run in SageMaker-managed VPCs

D. `ml-instance1` is run in the root user's VPCs

15. They wants to use the dataset in `db1` to train a model on `ml-instance2`. What is your recommendation?

A. Build a connection from `db1` to `ml-instance2`

B. Transform data from `db1` to S3 and provide the S3 location to `ml-instance2`

C. Move data to Amazon Redshift and build a connection between Redshift and `ml-instance2`

D. Move data to `ml-instance2` directly from `db1`

Answers to the practice questions

1. C

2. A

3. A

4. A

5. D

6. D

7. A

8. B

9. A

10. D

11. A

12. D

13. A

14. C

15. B

Further reading

For further insights into what you've learned in this chapter, refer to the following links:

- `https://aws.amazon.com/sagemaker/`
- `https://docs.aws.amazon.com/sagemaker/index.html`
- `https://aws.amazon.com/rekognition/`
- `https://aws.amazon.com/comprehend/`
- `https://aws.amazon.com/transcribe/`
- `https://aws.amazon.com/polly/`
- `https://aws.amazon.com/translate/`
- `https://aws.amazon.com/lex/`